

Blockchains and Smart Contracts

Term Assignment

1. CryptoSOS

You should write a smart contract in the Solidity language, to implement the “SOS” game in Ethereum.

The SOS Game

This classic game is similar to tic-tac-toe. It is played by two players, playing in turns. There is a board of 9 squares, organized in a 3x3 grid, which, in the context of this assignment will be enumerated as shown in Figure 1(a).

1	2	3
4	5	6
7	8	9

Fig 1(a)

O		S
O	O	S
		S

Fig 1(b)

O		S
O	O	S
S		S

Fig 1(c)

The squares are initially all empty. In each move, a player places exactly one **S** or **O** in an empty square. The game ends either when the SOS word is formed horizontally, vertically, or diagonally, or when there are no more vacant squares. The winner is the player who forms the SOS word. If the entire board gets filled up without the SOS word having been formed, it is considered a tie.

For instance, if a game is in the state of Figure 1(b) and the player whose turn is to play places an S in square 7, he forms the SOS word diagonally, as shown in Figure 1(c), and wins the game.

CryptoSOS API

In order to start playing, a player must call the `join()` function of CryptoSOS. Then he enters the waiting lobby, until a second player calls `join()` too, at which point a new game starts between these two players. If a third player calls `join()`, this will fail (i.e., `revert()` with a message that a game is already in progress). Of course, after a game has ended, it should be possible for any two players to join and start a new game.

Note that it is not possible for a player to play a game against himself!

Once a game has started and the two players have been registered, each player should call either `placeS(uint8)` or `placeO(uint8)`, to place an S or an O in a square of his choice. The parameter of these functions should, obviously, be an integer from 1 to 9, inclusive.

At any point, anyone may call the `getGameState()` function, which should return a 9-character-long string, in which each character is one of {-,S,O}, and signifies the state of the respective square, based on the aforementioned enumeration of Figure 1(a). Obviously, a

hyphen corresponds to an empty square. For instance, when called for a game that is in the state of Figure 1(b), it should return the string **O-SOOS--S**.

Participation fees and prizes

In order to join the game, a player should prepay precisely 1 ether when calling **join()**. When a game ends, the winner gets a prize of 1.8 ethers, while the remaining 0.2 ethers are kept by CryptoSOS. In case of a tie, each player gets 0.95 ethers back, while the remaining 1 ether is kept by CryptoSOS. The owner of CryptoSOS (i.e., the account that deployed the CryptoSOS smart contract) can claim the assets collected by CryptoSOS, by calling the **sweepBalance(uint amount)** function, which should send the requested amount to the owner's account, assuming the balance is sufficient, and that there is enough money left to pay off prizes or return amounts.

Events

When a player calls the **join()** function successfully (i.e., with the required payment), an event of type **StartGame(address, address)** is emitted, which reports the addresses of the two players starting a new game. More specifically, when the first player joins, this event is emitted with his address as the first argument, and the zero address as the second one. When the second player joins, a new event of this type is emitted, reporting both addresses. Such an event, with both addresses, effectively declares the beginning of a new game.

After every move, an event of type **Move(address, uint8, uint8)** is emitted. The first argument is the address of the player who made this move. The second argument is the number of the square where a letter was placed (1..9). The final argument is 1 for an S, and 2 for an O.

When a game ends with a winner, an event of type **Winner(address)** is emitted, reporting the address of the winner.

When a game ends with a tie, an event of type **Tie(address, address)** is emitted, reporting the addresses of both players, in the order they have registered in the first place.

Special Cases

If a player joins, and within 2 minutes no other player has joined, he is allowed to get a full reimbursement (1 ether) by calling the **cancel()** function.

While a game is in progress, if the player whose turn it is to play delays his move by 1 minute or more, the other player (i.e., the one who made the last move) is allowed to call the **tooslow()** function, and get 1.5 ether back, leaving the remaining 0.5 in the assets of the CryptoSOS smart contract, and terminating the game. In this case, the player who successfully called the **tooslow()** function is considered a winner, therefore the **Winner(address)** event should be emitted normally, as described earlier.

If none of the players have played for 5 minutes, then the owner is allowed to call **tooslow()**, ending the game ahead of time. This type of termination is considered a tie, therefore each player should get 0.95 ethers back and the respective **Tie(address, address)** event should be emitted, as described earlier.

2. General Comments

The methods described above form the smart contract's API, therefore should be public or external (you can decide what is appropriate). Of course, you are allowed to create any number of auxiliary methods which should be private or internal, however you should avoid declaring them as public/external. If you do so, please explain their functionality and the rationale behind your decision in the accompanying Readme file.

Errors pertaining to users' inputs should contain a short text message explaining the reason of the error (e.g., through `require()` or `revert()`).

Your implementation should aim at minimizing the gas costs, both at the deployment phase (i.e., try to keep your code as compact as possible) and at the execution phase (i.e., take care at storage and computation costs, avoid unnecessary loops, etc.).

Needless to say, one of the most important aspects is security. You should shield your smart contracts against any type of attacks, which we have studied in class. In your Readme file, explain the potential attacks you considered, and the steps you took to prevent them.

Your source code should be concise and clear regarding what it is doing and why. Use comments to describe what functions or parts of functions are doing. Add extra comments wherever you think your code is a bit difficult to understand, or performs something important (e.g., you are doing something intentionally to prevent some sort of attack or to save on gas costs, etc.).

3. Grading and Collaboration

The assignment has a weight of **2.5 out of 10** out of your final grade. The remaining weight of **7.5 out of 10** is allocated to the final exam.

The assignment is to be performed in **teams of two students**. Any collaboration between different teams, code that bares similarities, or implementations based on code you "borrowed" from others, will receive a flat zero. This concerns all parties, i.e., both the giver and taker of an implementation.

4. Submission Instructions

Send your implementation by email to voulgaris+sos@aueb.gr (necessarily this email address, else it will not be registered!) the smart contracts you have implemented as pure attached files (i.e., no zip, tar, wetransfer, or anything else), using the filenames **CryptoSOS.sol** and **Readme.txt** or **Readme.pdf** (please, no Word, or other formats), with a concise description of your overall design, the difficulties you had to deal with, the points that deserve extra mentioning, and the way you alleviated possible dangers of attacks.